

Department of Computer and Software Engineering
SE: Machine Learning

Course Instructor:	Date:
Day:	Semester:

Lab 8: Fraud Detection using Isolation Forest and Gaussian Density

Lab Title	CLO	BT	PLO	Weightage
Fraud Detection using Isolation Forest and Gaussian Density				

Name	Roll Number	Report /10	Viva /5	Total /15
Zunaira Abdul Aziz	BSSE23058			

Checked on: _____

Signature: _____

Objectives

- Understand anomaly detection in real-world fraud scenarios
- Implement Isolation Forest using sklearn
- Implement Gaussian density estimation from scratch
- Visualize anomalous regions

Problem Statement

You are designing a fraud detection system for a bank. Each transaction is represented using two features:

- Transaction Amount
- Time Gap Since Last Transaction

Normal transactions follow consistent patterns, while fraudulent transactions tend to deviate significantly.

Your task is to detect fraudulent transactions using:

- Isolation Forest (algorithmic approach)
- Gaussian Density Estimation (probabilistic approach)

Dataset Description

A synthetic dataset is provided:

- Normal data: Gaussian distribution
- Fraud data: Random outliers
- Labels:
 - 1 = Normal
 - -1 = Fraud

Part A: Isolation Forest (9 Marks)

Task 1: Train Isolation Forest

Implement a function to train an Isolation Forest model on the given dataset. The model isolates anomalies by recursively splitting the data using random feature selection and split values.

```
def fit_isolation_forest(X):  
    model = IsolationForest(contamination=0.11, random_state=42)  
    model.fit(X)  
    return model
```

Task 2: Predict Anomalies

Use the trained model to classify each transaction as normal or anomalous. Observe how many points are identified as anomalies and compare with the actual distribution.

```
def predict_iforest(model, X):  
    return model.predict(X)
```

Task 3: Analyze Anomaly Scores

Compute anomaly scores for all data points. These scores are related to the path length required to isolate a point in the trees. Points that are isolated quickly (short path length) are more likely to be anomalies.

```
def compute_iforest_scores(model, X):  
    return model.decision_function(X)
```

Part B: Gaussian Density Estimation (12 Marks)

Task 4: Estimate Gaussian Parameters

Compute the mean and covariance matrix of the dataset.

$$\mu = \frac{1}{N} \sum_{i=1}^N x_i$$

$$\Sigma = \frac{1}{N} \sum_{i=1}^N (x_i - \mu)(x_i - \mu)^T$$

These parameters represent the center and spread of the data

```
def compute_gaussian_params(X):
    mu = np.mean(X, axis=0)
    sigma = np.cov(X, rowvar=False)
    return mu, sigma.
```

Task 5: Compute Probability Density

Compute the probability density of each data point using the multivariate Gaussian distribution:

$$p(x) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu) \right)$$

Points far from the mean will have lower probability values.

```
def gaussian_density(X, mu, sigma):
    return multivariate_normal.pdf(X, mean=mu, cov=sigma)
```

Task 6: Detect Anomalies using Threshold

Classify transactions based on probability values:

If $p(x) < \epsilon \Rightarrow \text{Anomaly}$

Else $\Rightarrow \text{Normal}$

Experiment with different values of ϵ and observe how the classification changes.

```
def predict_gaussian(X, mu, sigma, threshold):
```

```
    p = gaussian_density(X, mu, sigma)
```

```
    predictions = np.ones(len(X))
```

```
    predictions[p < threshold] = -1
```

```
    return predictions
```

Part C: Visualization (6 Marks)

Task 7: Visualize Dataset

Plot the dataset in a two-dimensional space. Clearly distinguish between normal and anomalous transactions using different colors.

```
def plot_points(X, labels, title):
```

```
    plt.figure(figsize=(8, 6))
```

```
    plt.scatter(X[labels == 1, 0], X[labels == 1, 1], label="Normal", alpha=0.6)
```

```
    plt.scatter(X[labels == -1, 0], X[labels == -1, 1], label="Fraud", color="red")
```

```
    plt.xlabel("Transaction Amount")
```

```
    plt.ylabel("Time Gap")
```

```
    plt.title(title)
```

```
    plt.legend()
```

```
    plt.show()
```

Task 8: Plot Decision Boundaries

Visualize how both methods (Isolation Forest and Gaussian) separate normal data from anomalies. Compare the shape of the regions produced by each method





